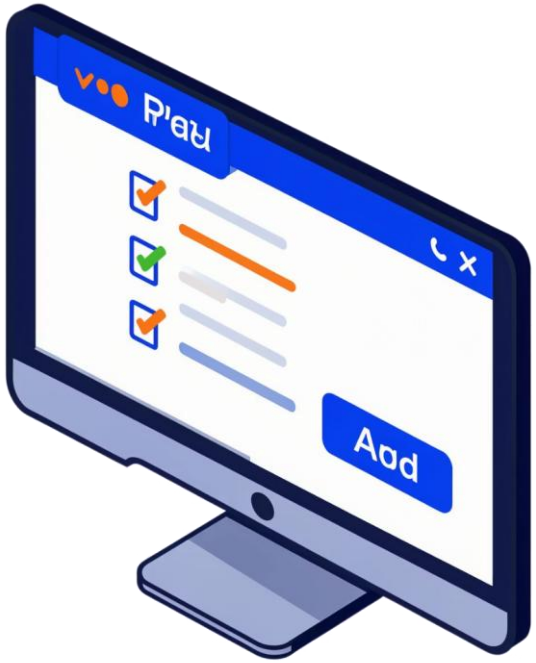
PYTHON PROJECT

Python To-Do List Application

A Simple Desktop Task Management System



Name : Parmar Aniket Ashvinkumar
Enroll No. : 12402080601003
Subject : Python

Organize daily tasks in one place

A record of daily activities

A To-Do List records and tracks daily activities.

A graphical interface

The application provides a graphical user interface.

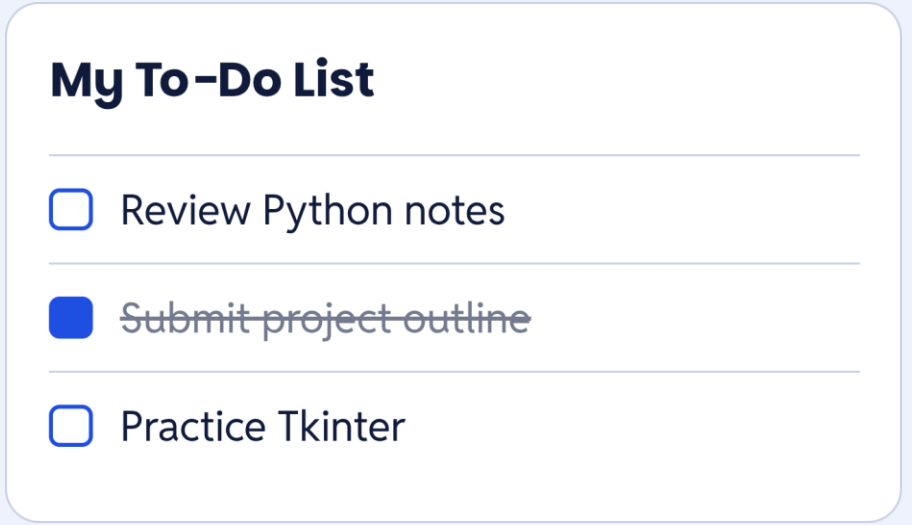
Pending or completed

Users manage pending and completed tasks.

Saved for later

Task information is saved locally for future use.

ONE PLACE



Tasks stay visible, with status easy to recognize.

The project focuses on simple, reliable task management

Designed for a beginner-friendly desktop experience.



Add and display

Add and display tasks easily.



Track status

Mark tasks as completed or pending.



Remove tasks

Delete unwanted tasks.



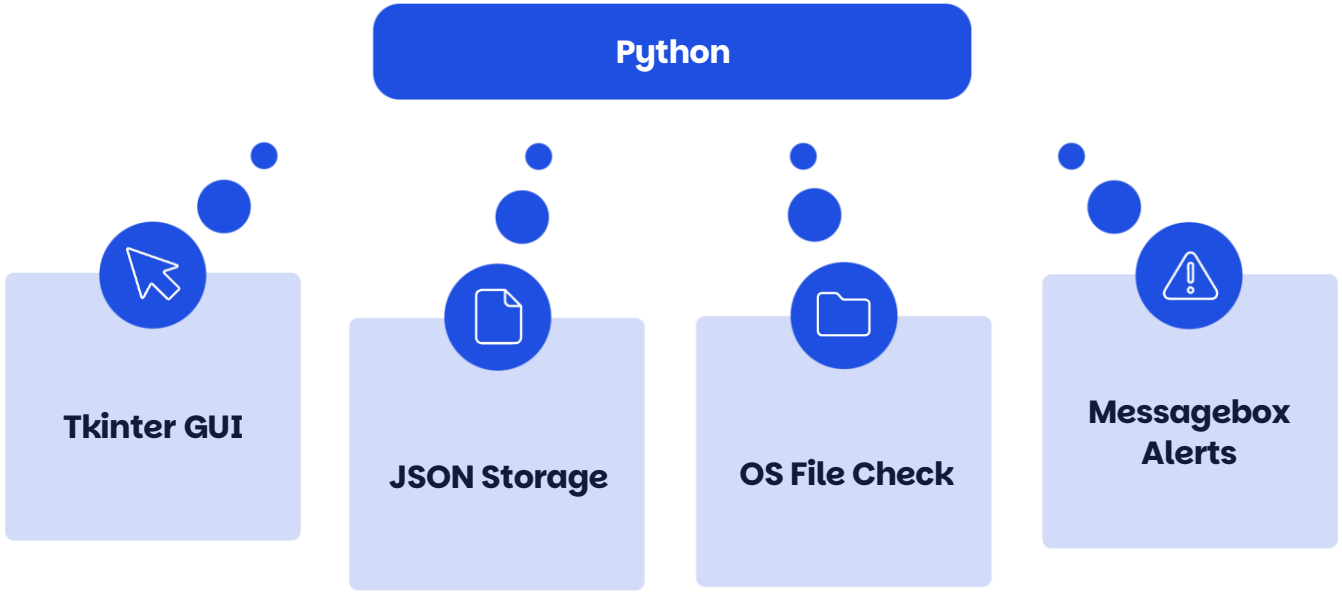
Keep progress

Remove completed tasks and save or reload tasks between sessions.

Five Python technologies support the application

Each technology has one clear responsibility.

- Application core
- User support



Five actions cover the daily task workflow

From entering a task to restoring it in the next session, the workflow stays compact.

01

Add Task

Add a new task to the list.

02

Mark Done / Undo

Change completion status.

03

Delete

Remove a selected task.

04

Clear Completed

Remove completed tasks.

05

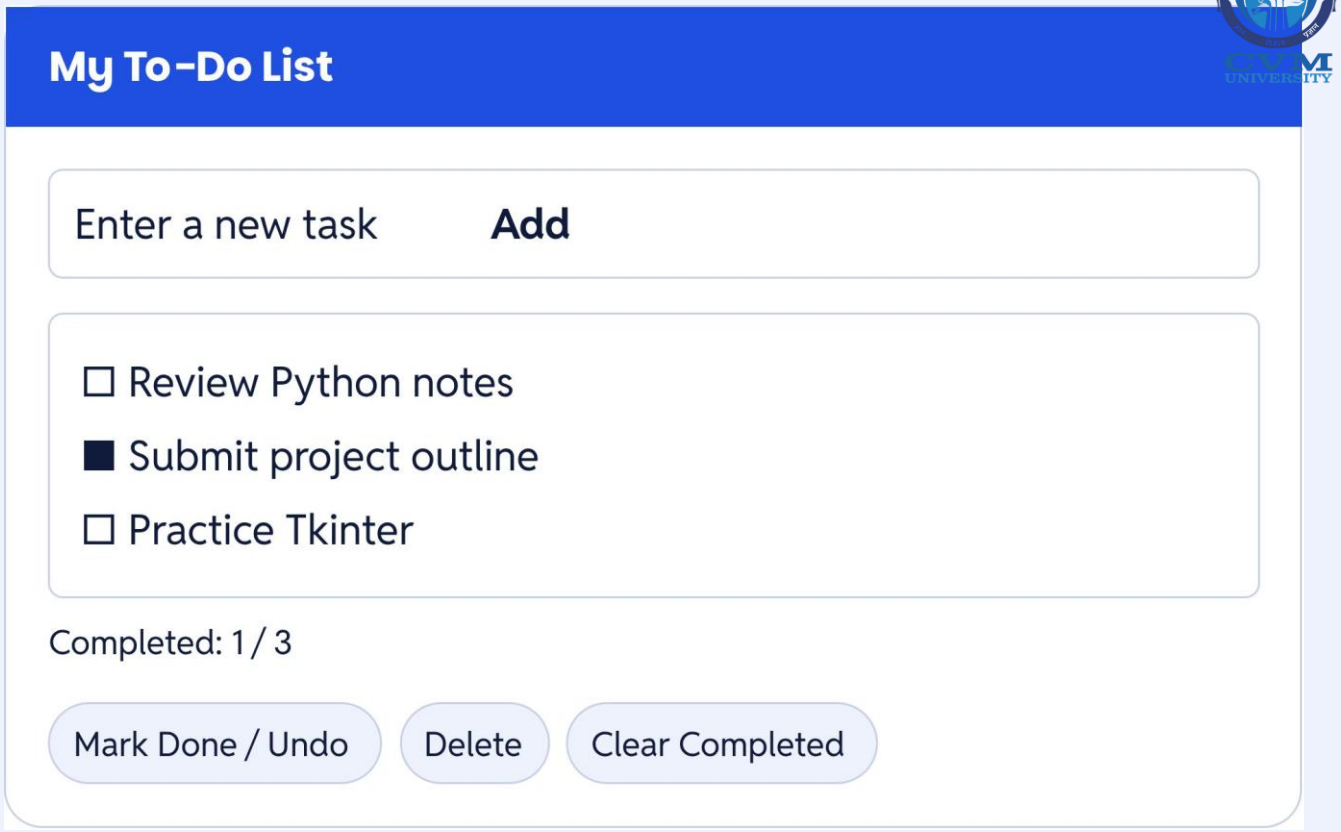
Save and Load

Store tasks in tasks.json and reload them at startup.

The interface keeps every action visible

Enter, inspect, update, and remove tasks in one window.

- **“My To-Do List”:** main window title.
- **Entry field + Add:** enter and add a new task.
- **List box:** displays all tasks.
- **Completion counter:** shows completed tasks out of total tasks.
- **Action buttons:** Mark Done / Undo, Delete, and Clear Completed.



Small functions keep the code organized

Each function owns one part of the task workflow.



load_tasks()

Loads saved tasks.



save_tasks()

Writes tasks to JSON.



refresh_tasks()

Updates the list and counter.



Task operations

add_task(), toggle_task(), delete_task(), clear_completed()

CODE EXCERPTS

```
def add_task():
    task = entry.get()
    if task:
        tasks.append(task)
        save_tasks()
        refresh_tasks()

def save_tasks():
    with open("tasks.json", "w") as file:
        json.dump(tasks, file)
```

Short functions make each responsibility easy to locate.

Each user action follows the same update loop

1 Start

2 Create GUI

3 Load Saved Tasks

4 Display Task List

5 Select an Action

6 Validate Input or Selection

7 Update Task List

8 Save to JSON

9 Refresh Display

10 Wait for Next Action

root.mainloop()

Keeps the GUI running and responsive.

JSON persistence

Preserves tasks between sessions.

Testing checks the implemented behavior

Test input	Expected output
Valid task added	It appears in the list.
Empty task added	Warning appears.
Task marked done	Status changes.
Selected task deleted	It is removed.
Clear completed	Only completed tasks are removed.
Restart application	Saved tasks load again.

EXPECTED OUTPUT

Review Python notes ✓

Practice Tkinter ✓

Completed: 2 / 2

Storage is local. No database, cloud sync, login, or deadlines are claimed.

CONCLUSION

A focused foundation for future improvements

Conclusion

Basic task management is handled through a simple GUI.

Tkinter provides the interface, JSON provides local persistence, and Python functions organize the program.

Thank You · Questions & Answers

Future scope

Add deadlines, priorities, reminders, categories, search, themes, or database and cloud storage.

